Data Loading, Storage and File Formats

Problem Statement: Analyzing Sales Data from Multiple File Formats

Dataset: Sales data in multiple file formats (e.g., CSV, Excel, JSON)

Description: The goal is to load and analyze sales data from different file formats, including CSV, Excel, and JSON, and perform data cleaning, transformation, and analysis on the dataset.

Tasks to Perform: Obtain sales data files in various formats, such as CSV, Excel, and JSON.

1. Load the sales data from each file format into the appropriate data structures or data frames.
2. Explore the structure and content of the loaded data, identifying any inconsistencies, missing values, or data quality issues.
3. Perform data cleaning operations, such as handling missing values, removing duplicates, or correcting inconsistencies.
4. Convert the data into a unified format, such as a common data frame or data structure, to enable seamless analysis
5. Perform data transformation tasks, such as merging multiple datasets, splitting columns, or deriving new variables.
6. Analyze the sales data by performing descriptive statistics, aggregating data by specific variables, or calculating metrics such as total sales, average order value, or product category distribution.
7. Create visualizations, such as bar plots, pie charts, or box plots, to represent the sales data and gain insights into sales trends, customer behavior, or product performance.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import json
```

1. `df_csv = pd.read_csv("___")`
   `df_excel = pd.read_excel("___")`
   `df_json = pd.read_json("___")`
2. `df_csv.info()`, `df_csv.describe()`, `df_csv.shape`, `df_csv.columns`
   `df_csv.isnull().sum()`
   `df_csv.duplicated().sum()`
   `df_csv['Gender'] = df_csv['Gender'].str.strip()` — Remove extra space
   ```
   for col in df_csv.columns:
       print(col, df_csv[col].unique())
   ```
   — Check inconsistent values
   `df_csv.dtypes` — data type
   `df[df['sales']<0]` — Check -ve values
3. `df.drop_duplicates(inplace=True)`
   `df_csv['Sales'] = df_csv['Sales'].fillna(df['Sales'].mean())`
   `df_csv['Gender'] = df_csv['Gender'].fillna(df['Gender'].mediun()[0])`
4. `df_csv = pd.DataFrame(df_csv)`
   `df_excel = pd.DataFrame(df_excel)`
   `df_json = pd.Dataframe(df_json)`
   `df = pd.concat([df_csv, df_excel, df_json], ignore_index=True)`
5. `df[['Branch Code']['Invoice No']] = df['Invoice ID'].str.split('-', n=1, expand=1)`
   `df['Date'] = pd.to_datetime(df['Date'])`
   `df['Year'] = df['Date'].dt.year`
   `df['month'] = df['Date'].dt.month`
   `df['calculated Amount'] = df['Unit price'] * df['Quantity']`
6. `df.describe()`
   `df.groupby('Product line')['Total'].sum()`
   `df['Total'].mean()`
   `df['Product line'].value_counts()`
7. `region_sales = df_excel.groupby('Region')['Sales'].sum()`
   `region_sales.plot(kind='bar', figsize=(10,6))`

   `region_sales.plot(kind='pie', autopct='%1.1f%%')`

   `plt.boxplot(df_excel['sales'].head(10))`
   `plt.show()`

Data Cleaning and Preparation

Problem Statement: Analyzing Customer Churn in a Telecommunications Company

Dataset: "Telecom_Customer_Churn.csv"

Description: The dataset contains information about customers of a telecommunications company and whether they have churned (i.e., discontinued their services). The dataset includes various attributes of the customers, such as their demographics, usage patterns, and account information. The goal is to perform data cleaning and preparation to gain insights into the factors that contribute to customer churn.

Tasks to Perform:

1. Import the "Telecom_Customer_Churn.csv" dataset.
2. Explore the dataset to understand its structure and content.
3. Handle missing values in the dataset, deciding on an appropriate strategy.
4. Remove any duplicate records from the dataset.
5. Check for inconsistent data, such as inconsistent formatting or spelling variations, and standardize it.
6. Convert columns to the correct data types as needed.
7. Identify and handle outliers in the data.
8. Perform feature engineering, creating new features that may be relevant to predicting customer churn.
9. Normalize or scale the data if necessary.
10. Split the dataset into training and testing sets for further analysis.
11. Export the cleaned dataset for future analysis or modeling.

```
import pandas as pd
import matplotlib.pyplot as plt
From sklearn.preprocessing import MinMaxScaler, StandardScaler
From sklearn.model_selection import train_test_split
df = pd.read_csv("____")
df.head(), df.tail(), df.dtypes,
df.unique(), df.info(), df.describe(),
df.columns, df.shape,
```

```
3. df.isnull().sum()
   for col in df.columns:
       if df[col].dtype == 'object':
           df[col] = df[col].fillna(df[col].mode()[0])
       else:
           df[col] = df[col].fillna(df[col].mean())
   print(df.isnull().sum().sum())
4. df.duplicated().sum()
   df.drop_duplicates()
5. for col in df.columns:
       if df[col].dtype == 'object':
           print(col, df[col].unique())
   for col in df.columns:
       if df[col].dtype == 'object':
           df[col] = df[col].str.strip()
6. df['Gender'] = df['Gender'].astype(str)
7. plt.figure(figsize=(10,8))
   plt.boxplot(x = df['Total Revenue'])
   plt.show()
Handle → Q1 = df['____'].quantile(0.25)
   Q3 = df['____'].quantile(0.75)
   IQR = Q3 - Q1
   lower = Q1 - 1.5 * IQR
   upper = Q3 + 1.5 * IQR
   df['____'] = df['____'].clip(lower, upper)
   OR outliers = df[(df['____'] <= lower) | (df['____'] >= upper)]
8. df['Total'] = df['No. of Referrals'] * df['Monthly Charges']
9. Scalar = MinMaxScalar()
   df[['Total']] = Scalar.fit_transform(df[['Total']])
10. X_train, X_test, Y_train, Y_test = train_test_split(data, testSize=0.25, random_state=42)
11. df.to_csv('____', index=False)
    print(os.getcwd())
```

4

Data Wrangling

Problem Statement: Data Wrangling on Real Estate Market

Dataset: "RealEstate_Prices.csv"

Description: The dataset contains information about housing prices in a specific real estate market. It includes various attributes such as property characteristics, location, sale prices, and other relevant features. The goal is to perform data wrangling to gain insights into the factors influencing housing prices and prepare the dataset for further analysis or modeling.

Tasks to Perform:

1. Import the "RealEstate_Prices.csv" dataset. Clean column names by removing spaces, special characters, or renaming them for clarity.
2. Handle missing values in the dataset, deciding on an appropriate strategy (e.g., imputation or removal).
3. Perform data merging if additional datasets with relevant information are available(e.g., neighborhood demographics or nearby amenities).
4. Filter and subset the data based on specific criteria, such as a particular time period, property type, or location.
5. Handle categorical variables by encoding them appropriately (e.g., one-hot encoding or label encoding) for further analysis.
6. Aggregate the data to calculate summary statistics or derived metrics such as average sale prices by neighborhood or property type.
7. Identify and handle outliers or extreme values in the data that may affect the analysis or modeling process.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing
  import LabelEncoder, OneHotEncoder
```

```
1. df.columns = df.columns.str.strip().str.lower().str.replace
                (r'[^a-z0-9]+','_',regex=T)
2. df.isnull().sum()
   df.duplicated().sum()
3. df_1 = pd.DataFrame({'no':[1,2,3], 'N':['A','B','C']})
   df_2 = pd.merge(df, df_1, on:'no')
4. Filter_loc = df[df['location']=='Pune']
   Filter_data = df[df['Price']>40]
   Filter_data = df[(df['price']>40) & (df['Price']<80)]
   Subset_data = df[['loc','Price']]
5. df['Age'] = df['house age'].apply(lambda x: 'old' if x>30
   df.head()                                    else 'New')
   labelEncoder = labelEncoder()
   df['__'] = labelEncoder.fit_transform(df['__'])
   df.head()
6. avg_price = df.groupby('__')['__'].mean()
                                    .sum()
                                    .value_count()
7. plt.figure(figsize=(10,8))
   plt.boxplot(df['___'].head(_))
   plt.show()
   Q1 = df['__'].quantile(0.25)
   Q3 = df['__'].quantile(0.75)
   IQR = Q3-Q1
   lower = Q1 - 1.5 * IQR
   upper = Q3 + 1.5* IQR
   df['__'] = df['__'].clip(lower,upper)

   outliers = df[(df['___']<lower)|(df['__']>upper)]
   print outliers
```

Data Visualization using matplotlib

5

Problem Statement: Analyzing Air Quality Index (AQI) Trends in a City

Dataset: "City_Air_Quality.csv"

Description: The dataset contains information about air quality measurements in a specific city over a period of time. It includes attributes such as date, time, pollutant levels (e.g., PM2.5, PM10, CO), and the Air Quality Index (AQI) values. The goal is to use the matplotlib library to create visualizations that effectively represent the AQI trends and patterns for different pollutants in the city.

Tasks to Perform:

1. Import the "City_Air_Quality.csv" dataset.
2. Explore the dataset to understand its structure and content.
3. Identify the relevant variables for visualizing AQI trends, such as date, pollutant levels, and AQI values.
4. Create line plots or time series plots to visualize the overall AQI trend over time.
5. Plot individual pollutant levels (e.g., PM2.5, PM10, CO) on separate line plots to visualize their trends over time.
6. Use bar plots or stacked bar plots to compare the AQI values across different dates or time periods.
7. Create box plots or violin plots to analyze the distribution of AQI values for different pollutant categories.
8. Use scatter plots or bubble charts to explore the relationship between AQI values and pollutant levels.
9. Customize the visualizations by adding labels, titles, legends, and appropriate color schemes.

3. R_variable = df[['Date', 'PM2.5', 'PM10', 'CO', 'O3', 'AQI']]

4. plt.figure(figsize=(10,6))
   plt.plot(df['Date'].head(40), df['AQI'].head(40))
   plt.show()

5. plt.figure(figsize=(15,8))
   plt.subplot(3,1,1)
   plt.plot(df2['Date'], df2['PM2.5']
   :
   plt.tight_layout()
   plt.show

6. plt.box(df['Date'].head(30), df['AQI'].head(30)

7. df2 = df[['__', '__', '__', '__']].dropna().head()
   plt.figure(figsize=(10,5))
   plt.boxplot([df['AQI']*5])
   plt.xticks([1,2,3,4,5], ['__', '__', '__', '__', '__'])
   plt.show()

8. plt.scatter(df['__'], df['__']

Data Aggregation 6

Problem Statement: Analyzing Sales Performance by Region in a Retail Company

Dataset: "Retail_Sales_Data.csv"

Description: The dataset contains information about sales transactions in a retail company. It includes attributes such as transaction date, product category, quantity sold, and sales amount. The goal is to perform data aggregation to analyze the sales performance by region and identify the top-performing regions.

Tasks to Perform:

1. Import the "Retail_Sales_Data.csv" dataset.
2. Explore the dataset to understand its structure and content.
3. Identify the relevant variables for aggregating sales data, such as region, sales amount, and product category.
4. Group the sales data by region and calculate the total sales amount for each region.
5. Create bar plots or pie charts to visualize the sales distribution by region.
6. Identify the top-performing regions based on the highest sales amount.
7. Group the sales data by region and product category to calculate the total sales amount for each combination.
8. Create stacked bar plots or grouped bar plots to compare the sales amounts across different regions and product categories

Data aggregation is the process of collecting & combining data from diff records to get a summary or useful info.

Get totals, avg, count, summaries..

() — call a fun / method

[] — select a coln / access data

Region → Gender

SalesAmount → Total Amt

Product Category → — // —

```
1. import pandas as pd
   import matplotlib.pyplot as plt
2. df = pd.read_csv(r" ____ ")
   df.head()
3. df.info(), df.describe(), df.shape, df.columns
4. df = df[['Region', 'Sales Amount', 'Product Category']]
   df.head()
5. region_sales = df.groupby('Region')['Sales Amount'].sum().reset_index()
   print(region_sales)
6. plt.figure(figsize=(8,5))
   plt.bar(region_sales['Region'], region_sales['Sales Amount'])
   plt.title('Sales Distribution by Region')
   plt.xlabel('Region')
   plt.ylabel('Total Sales Amount')
   plt.show()                                                        } bar plot
   plt.figure(figsize=(7,7))
   plt.pie(region_sales['Sales Amount'], labels=region_sales['Region'],
           autopct='%1.1f%%')                                        } pie chart
   plt.title('Sales Distribution by Region')
   plt.show()
7. top_region = region_sales.sort_values(
           by='Sales_Amount', ascending=False)
   print(top_region) / print(top_region.head(1))
8. category_sales = df.groupby(
           ['region', 'Product Category'])['Sales Amount'].sum().unstack()
   print(category_sales)
9. category_sales.plot(
     kind='bar', stacked=True, figsize=(10,6))
   plt.title('Sales by Region & Product Category')
   plt.xlabel('Region')
   plt.ylabel('Total Sales Amount')
   plt.legend(title='Product Category')
   plt.show
```